

LAB REPORT

Dice Game

3220102382 潘谷雨

2024.12.19

1. Problem Description

This exercise is to design an electronic dice game. This game is popularly known as craps in the United States. The game involves two dice, each of which can have a value between 1 and 6. Two counters are used to simulate the roll of the dice. Each counter counts in the sequence 1, 2, 3, 4, 5, 6, 1, 2, . . . Thus, after the “roll” of the dice, the sum of the values in the two counters will be in the range 2 through 12. The rules of the game are as follows.

1. After the first roll of the dice, the player wins if the sum is 7 or 11. The player loses if the sum is 2, 3, or, 12. Otherwise, the sum the player obtained on the first roll is referred to as a point, and he or she must roll the dice again.
 2. On the second or subsequent roll of the dice, the player wins if the sum equals the point, and he or she loses if the sum is 7. Otherwise, the player must roll again until he or she finally wins or loses.
- Figure 1 shows the block diagram for the dice game. The inputs to the dice game come from two push buttons, Rb (roll button) and Reset. Reset is used to initiate a new game. When the roll button is pushed, the dice counters count at a high speed, so the values cannot be read on the display. When the roll button is released, the values in the two counters are displayed via 7-segment displays.

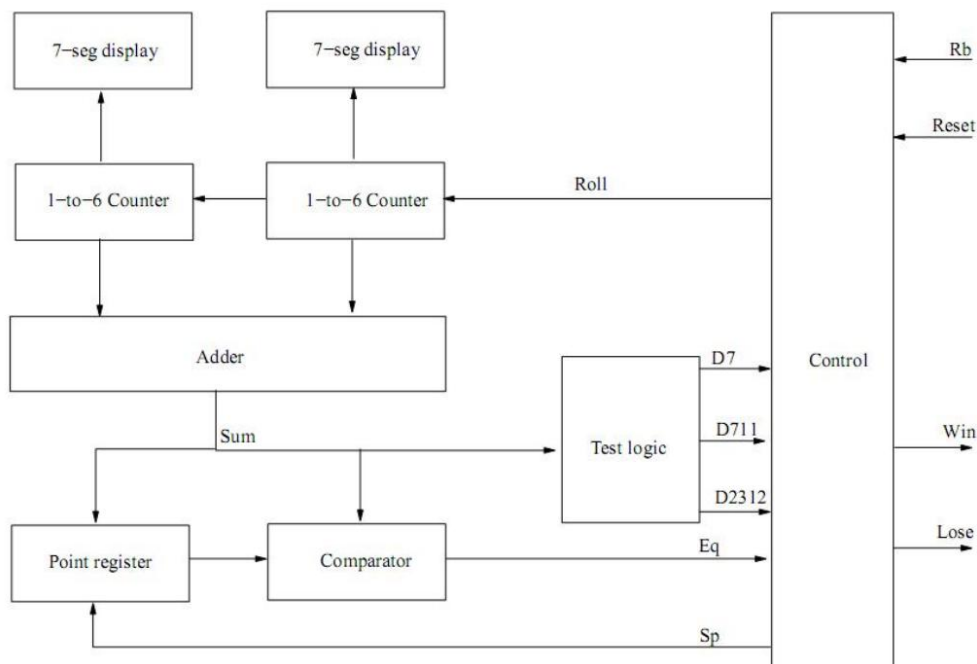


Figure 1. Block diagram for dice game.

The components for the dice game shown in the block diagram (Figure 1) include an adder, which adds the two counter outputs, a register to store the point, test logic to determine conditions for win or lose, and a control circuit. Input signals to the control circuit are defined as follows:

D7 = 1 if the sum of the dice is 7

D711 = 1 if the sum of the dice is 7 or 11

D2312 = 1 if the sum of the dice is 2, 3, or 12

Eq = 1 if the sum of the dice equals the number stored in the point register

Rb = 1 when the roll button is pressed

Reset = 1 when the reset button is pressed

Outputs from the control circuit are defined as follows:

Roll = 1 enables the dice counters

Sp = 1 causes the sum to be stored in the point register

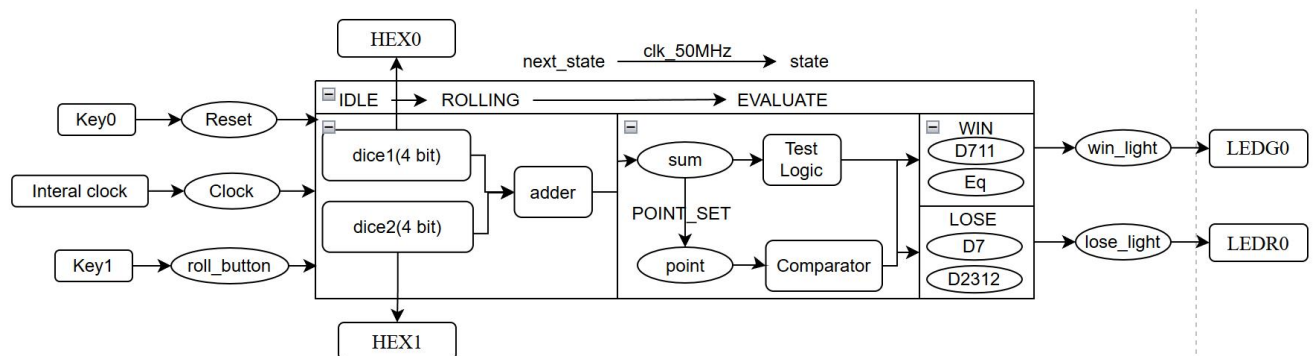
Win = 1 turns on the win light

Lose = 1 turns on the lose light

Design and implement the dice game shown in Figure 1 using VHDL code as follows:

1. Create a new Quartus II project for this exercise.
2. Generate the required VHDL file, include it in your project.
3. Compile the circuit and use functional simulation to verify the correct operation of your designed dice game.
4. Use the displays HEX1 and HEX0 to show the values of two counter outputs. Also, use push button KEY0 for Reset and KEY1 for the roll button. If the player wins, light on a green LED light on the board, and if the player loses, light on a red LED light on the board.
5. Add to your project the necessary pin assignments for the DE2 board. Compile the circuit and download it into the FPGA chip.
6. Test the functionality of your design by playing the dice game.

2. Design Formulation



Pic.1 design formulation of dice game

Pic.1 shows a block diagram of a Finite State Machine (FSM) design for simulating a dice game. The FSM has multiple states including IDLE, ROLLING, EVALUATE, WIN, LOSE, and POINT_SET.

The system starts in the IDLE state, waiting, and the user starts the game by pressing Key1 (roll_button). The system detects the signal from the roll_button and moves from the IDLE state to the ROLLING state. In the ROLLING state, the two dice (dice1 and dice2) start rolling, generating random numbers while the system calculates the sum of values of the two dice. The

values are displayed via HEX0 and HEX1. Release the button, the dice stop rolling and the system enters the EVALUATE state, if the sum is 7 or 11, the player wins directly (WIN state), if the sum is 2, 3 or 12, the player loses the game directly (LOSE state), otherwise the system records the value as points and enters the POINT_SET state, waiting for the user to roll the dice again.

After the user presses the roll button, the system returns to the ROLLING state and starts a new round of dice rolling. If the sum of the dice rolled again in the POINT_SET state is equal to the number of points, the player wins (WIN state), and if the sum of the dice rolled during the attempt to reach the number of points is 7, the player loses the game (LOSE state).

Depending on the outcome of the game, the appropriate LED (win_light or lose_light) will light up to indicate whether the player has won or lost. The user can reset the game by pressing Key0 (Reset) to return the system to the IDLE state, ready to start a new round of the game.

3. Design Entry

seven_segment_decoder.vhd:

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity seven_segment_decoder is
6  Port ( bin : in  STD_LOGIC_VECTOR(3 downto 0);
7        seg : out STD_LOGIC_VECTOR(6 downto 0));
8  end seven_segment_decoder;
9
10 architecture Behavioral of seven_segment_decoder is
11 begin
12     process(bin)
13     begin
14         case bin is
15             when "0000" => seg <= "1000000"; -- 0
16             when "0001" => seg <= "1111001"; -- 1
17             when "0010" => seg <= "0100100"; -- 2
18             when "0011" => seg <= "0110000"; -- 3
19             when "0100" => seg <= "0011001"; -- 4
20             when "0101" => seg <= "0010010"; -- 5
21             when "0110" => seg <= "0000010"; -- 6
22             when "0111" => seg <= "1111000"; -- 7
23             when "1000" => seg <= "0000000"; -- 8
24             when "1001" => seg <= "0010000"; -- 9
25             when "1010" => seg <= "0001000"; -- A
26             when "1011" => seg <= "0000011"; -- B
27             when "1100" => seg <= "1000110"; -- C
28             when "1101" => seg <= "0100001"; -- D
29             when "1110" => seg <= "0000110"; -- E
30             when others => seg <= "0001110"; -- F
31         end case;
32     end process;
33 end Behavioral;

```

clock_divider.vhd:

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity clock_divider is
6  Port ( clk_50MHz : in  STD_LOGIC;
7        reset : in  STD_LOGIC;
8        clk : out  STD_LOGIC);
9  end clock_divider;

```

```

11 architecture Behavioral of clock_divider is
12     signal count : unsigned(25 downto 0) := (others => '0');
13 begin
14     process(clk_50MHz, reset)
15     begin
16         if reset = '0' then
17             count <= (others => '0');
18             clk <= '0';
19         elsif rising_edge(clk_50MHz) then
20             if count = 120 - 1 then--100 - 1 then--
21                 count <= (others => '0');
22                 clk <= '1';
23             else
24                 count <= count + 1;
25                 if count = 60 - 1 then--50 - 1 then--
26                     clk <= '0';
27                 end if;
28             end if;
29         end if;
30     end process;
31 end Behavioral;

```

dice_game.vhd:

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity dice_game is
6  Port (
7      clk_50MHz : in STD_LOGIC;
8      reset : in STD_LOGIC;
9      roll_button : in STD_LOGIC;
10     win_light : out STD_LOGIC;
11     lose_light : out STD_LOGIC;
12     dice1_disp : out STD_LOGIC_VECTOR(6 downto 0);
13     dice2_disp : out STD_LOGIC_VECTOR(6 downto 0);
14     clk : buffer STD_LOGIC;
15     dice1, dice2 : buffer unsigned(3 downto 0) := "0001";
16     sum : buffer unsigned(3 downto 0);
17     Roll, Sp: buffer STD_LOGIC;
18     point : buffer unsigned(3 downto 0) := "0000";
19     next_state_leds: buffer STD_LOGIC_VECTOR(2 downto 0);
20     state_leds : out STD_LOGIC_VECTOR(2 downto 0)
21 );
22 end dice_game;
23
24 architecture Behavioral of dice_game is
25     type state_type is (IDLE, ROLLING, EVALUATE, WIN, LOSE, POINT_SET);
26     signal state, next_state : state_type := IDLE;
27     signal pre_roll_button : STD_LOGIC;
28
29     component clock_divider is
30     Port (
31         clk_50MHz : in STD_LOGIC;
32         reset : in STD_LOGIC;
33         clk : out STD_LOGIC
34     );
35     end component;
36
37     component seven_segment_decoder is
38     Port (
39         bin : in STD_LOGIC_VECTOR(3 downto 0);
40         seg : out STD_LOGIC_VECTOR(6 downto 0)
41     );
42     end component;
43
44 begin
45     U_clk_divider: clock_divider port map (clk_50MHz, reset, clk);

```

```

47 process(clk, reset)
48 begin
49     if reset = '0' then
50         state <= IDLE;
51     elsif rising_edge(clk) then
52         state <= next_state;
53         pre_roll_button <= roll_button;
54         if Sp= '1' then point <= sum; end if;
55     end if;
56 end process;
57
58 process(state, roll_button, reset, sum)
59 begin
60     Roll <= '0';
61     Sp <= '0';
62     win_light <= '0';
63     lose_light <= '0';
64     case state is
65         when IDLE =>
66             if roll_button = '0' then
67                 next_state <= ROLLING;
68             else
69                 next_state <= IDLE;
70             end if;
71
72         when ROLLING =>
73             if roll_button = '0' then
74                 Roll <= '1';
75                 next_state <= ROLLING;
76             elsif sum = 7 or sum = 11 then
77                 next_state <= WIN;
78             elsif sum = 2 or sum = 3 or sum = 12 then
79                 next_state <= LOSE;
80             else
81                 Sp <= '1';
82                 next_state <= POINT_SET;
83             end if;
84
85         when WIN =>
86             win_light <= '1';
87             if reset = '0' then
88                 next_state <= IDLE;
89             end if;
90
91         when LOSE =>
92             lose_light <= '1';
93             if reset = '0' then
94                 next_state <= IDLE;
95             end if;
96
97         when POINT_SET =>
98             if roll_button = '0' then
99                 Roll <= '1';
100                 next_state <= POINT_SET;
101             elsif pre_roll_button = '0' and sum = point then
102                 next_state <= WIN;
103             elsif sum = 7 then
104                 next_state <= LOSE;
105             else
106                 next_state <= POINT_SET;
107             end if;
108
109         when others =>
110             next_state <= IDLE;
111     end case;
112 end process;

```



```

114 process (clk, reset)
115     variable temp_dice1 : unsigned(3 downto 0);
116     variable temp_dice2 : unsigned(3 downto 0);
117 begin
118     if reset = '0' then
119         dice1 <= "0001";
120         dice2 <= "0001";
121     elsif Roll = '1' and rising_edge(clk) then
122         temp_dice1 := dice1 + 1;
123         temp_dice2 := dice2;
124         if temp_dice1 > 6 then
125             temp_dice1 := "0001";
126             temp_dice2 := temp_dice2 + 1;
127         end if;
128         sum <= temp_dice1 + temp_dice2;
129         dice1 <= temp_dice1;
130         dice2 <= temp_dice2;
131     end if;
132 end process;

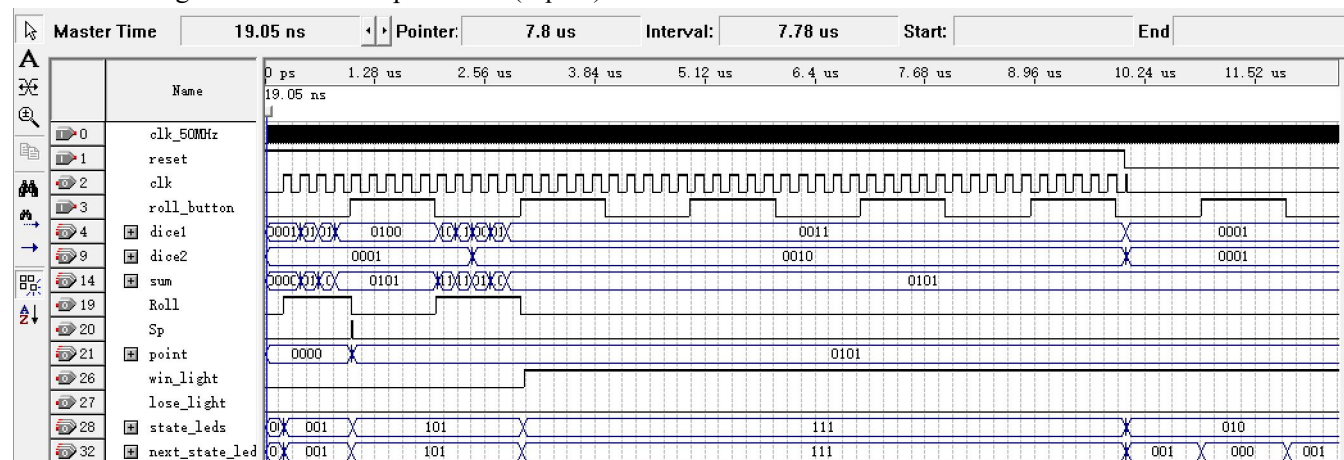
134 -- Instantiate the Seven Segment Decoders for both dice
135 U_dice1_decoder: seven_segment_decoder port map (bin => std_logic_vector(dice1), seg => dice1_disp);
136 U_dice2_decoder: seven_segment_decoder port map (bin => std_logic_vector(dice2), seg => dice2_disp);

138 process(state, next_state)
139 begin
140     case state is
141         when IDLE => state_leds <= "000";
142         when ROLLING => state_leds <= "001";
143         when EVALUATE => state_leds <= "011";
144         when WIN => state_leds <= "111";
145         when LOSE => state_leds <= "100";
146         when POINT_SET => state_leds <= "101";
147     end case;
148
149     case next_state is
150         when IDLE => next_state_leds <= "000";
151         when ROLLING => next_state_leds <= "001";
152         when EVALUATE => next_state_leds <= "011";
153         when WIN => next_state_leds <= "111";
154         when LOSE => next_state_leds <= "100";
155         when POINT_SET => next_state_leds <= "101";
156     end case;
157 end process;
158 end Behavioral;

```

4. Simulation and Synthesis Results

Roll the dice again and sum end up the same($Eg = 1$):

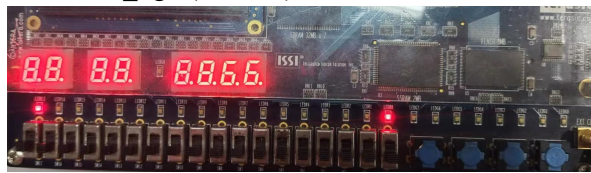


D711 = 1:

5. Experimental Results

(1) $D_{2312} = 1$

See Pic.2 When the first roll of the dice ends up in “6” and “6”, the sum is assigned “12”, which means $D_{2312} = 1$ and lose_light(LED_{R0}) is on.



Pic.2

(2) $D_{711} = 1$

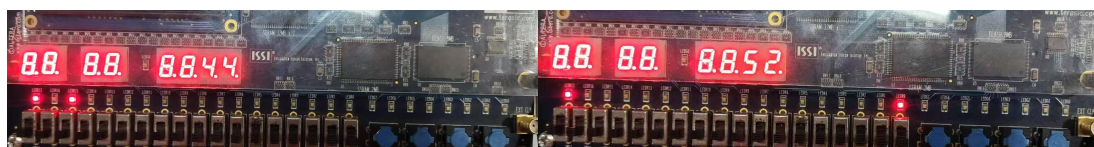
See Pic.3. When the first roll of the dice ends up in “2” and “5”, the sum is assigned “7”, which means $D_{711} = 1$ and win_light(LED_{G0}) is on.



Pic.3

(3) $D_7 = 1$

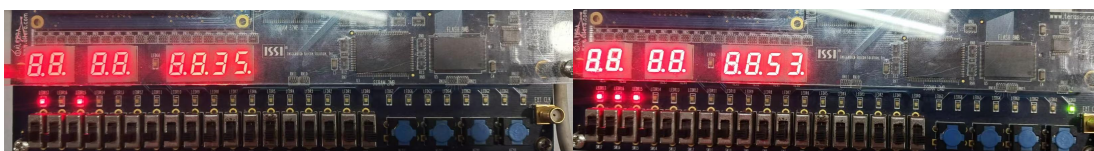
See Pic.4. When the first roll of the dice ends up in “4” and “4”, the sum is assigned “8”, which means state of the roll-dice system enters “POINT_SET”, waiting for next roll. After several rolls, when the dice ends up in “5” and “2”, the sum is assigned “7”, making $D_7 = 1$ and lose_light(LED_{R0}) is on.



Pic.4

(4) $E_q = 1$

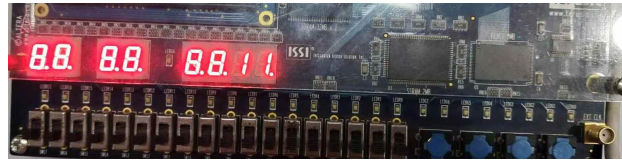
See Pic.5. When the first roll of the dice ends up in “3” and “5”, the sum is assigned “8”, which means state of the roll-dice system enters “POINT_SET”, waiting for next roll. After several rolls, when the dice ends up in “5” and “3”, the sum is assigned “8”, which equals to the first roll, making $E_q = 1$ and win_light(LED_{G0}) is on.



Pic.5

(5) reset = 0

After result of the game is decided, press Key0 and reset the



Pic.5

6. Discussion and Conclusion

In the course of the dice_game project, I encountered some problems related to the characteristics of the VHDL language, especially the delay characteristics of the signal assignment and the timing problems encountered in the simulation. Through this project, I have gained a deeper understanding of VHDL design and accumulated valuable experience.

Signal assignment in VHDL is inert, i.e. when we use “signal <= value” to make an assignment in a process, this assignment will not take effect immediately, but will update the value of the signal at the end of the current simulation time point and before the next simulation time point. This leads to a situation where the old value may be read when the signal is first assigned a value in a process. For example, If dice1 is compared to 6 after directly adding by 1, a situation where dice1=0111 will occur. By contrast, the assignment of variables is immediate and ensures that all operations are performed at the same point in time based on the latest value, so the variable temp_dice1 is used for intermediate calculations.

Also, we should try to keep the data types of all relevant signals and variables consistent to minimize unnecessary type conversions. For example, define sum, dice1 and dice2 as unsigned instead of mixing integer and unsigned, which can improve computation efficiency and reduce processing time.

During the simulation process, I also encountered the problem that some operations could not be completed in one clock cycle due to the short crossover time. This problem is especially evident when complex calculations or multiple state transitions are involved. It is therefore important to plan the amount of tasks per clock cycle to ensure that critical operations can be completed in a single clock cycle. This can be done by splitting complex computational tasks, simplify the logic structure of the state machine as much as possible, and optimizing algorithms to reduce the number of computational steps required. And we should consider asynchronous event processing. For external input signals such as buttons, they should be sampled on the clock edge, and in addition, the corresponding action should be triggered only when a change in the button state (e.g., from released to pressed) is detected, instead of directly relying on the immediate state of the button signal.

Through this dice_game project, I have gained a deep understanding of the importance of signal assignment delay characteristics in VHDL design and how to avoid these problems through reasonable programming practices. At the same time, I also realized the irreplaceable role of simulation verification in the process of hardware description language development. These experiences and lessons will have a positive impact on my future FPGA development work and help me build more efficient and reliable digital systems.